Agent Postmortem Generator

Part 2: Parser & Validation Engine

One-line purpose: Convert raw JSON trace input into a structurally valid, diagnostics-rich trace document that downstream modules can trust.

---

1. Title Page

Field	Value
Project Name	Agent Postmortem Generator
Part Number	2
Module Name	Parser & Validation Engine
Purpose	Convert raw JSON trace input into a structurally valid, diagnostics-rich trace document that downstream modules can trust.

---

2. Executive Overview

The Parser & Validation Engine is the first executable trust boundary in the pipeline. It accepts raw input for the MVP's generic JSON trace format, decodes it into structured data, validates it against the project's input contract, and produces a validated trace document with precise diagnostics.

This module exists because downstream modules must not be forced to guess whether the input is malformed, incomplete, or semantically inconsistent. The canonical trace model, analysis engine, and postmortem generator all assume they are operating on a document that has already passed structural and semantic checks. That assumption is only safe if the parser and validation layer is strict, deterministic, and explicit about failures.

In the system architecture, this module sits between the Input layer and the Normalizer. It does not normalize framework-specific concepts. It does not infer causality. It does not repair traces. It only decides whether the input is valid enough to proceed and, if not, why not.

```
Input
  |
  v
Parser + Validation Engine
```

```
  |
  v
Validated Generic Trace Document
  |
  v
Normalizer
```

---

3. Problem Statement

Raw agent traces are messy by default. They may arrive as files, strings, command output, or captured logs. They may contain malformed JSON, missing fields, invalid timestamps, duplicate identifiers, out-of-order records, or partially written runs. If these issues are not isolated early, every downstream module becomes more complex, less deterministic, and harder to test.

This problem is hard for three reasons:

1. **JSON validity is not enough.** A payload can be syntactically valid JSON and still be unusable as a trace.
2. **Trace shape varies in subtle ways.** Even within the MVP's single generic JSON format, producers may omit optional fields, reorder event arrays, or include unknown metadata.
3. **Failure handling must remain deterministic.** The same bad input must always produce the same diagnostics in the same order. Otherwise debugging the debugging tool becomes the new hobby.

The MVP needs this module because without a strict parser and validation layer, the rest of the pipeline would either accept garbage or start accumulating defensive logic. That is how simple systems rot into small horror museums.

---

4. Goals

What success looks like

- The module accepts the MVP trace input format from local sources.
- It rejects malformed input early and reports precise, actionable diagnostics.
- It preserves enough raw structure and source location metadata for downstream debugging.
- It performs only validation, not normalization beyond trivial type-safe decoding.
- It behaves deterministically for identical inputs.
- It is easy to unit test with fixture files.

What this module must achieve

- Parse a single JSON document into an in-memory structured representation.
- Validate required top-level fields, event-level fields, and trace-level constraints.
- Produce a stable diagnostics list with severity, location, and message.
- Return a validated trace document only when fatal errors are absent.
- Preserve unknown fields for downstream inspection where safe and useful.
- Provide a clean boundary that downstream code can trust.

---

5. Non-Goals

This module explicitly does **not**:

- Normalize the trace into the canonical model.
- Reconstruct causality, failure chains, or root cause.
- Interpret framework-specific semantics from LangGraph, CrewAI, OpenAI Agents SDK, AutoGen, or future adapters.
- Repair malformed traces automatically.
- Fetch missing data from the network.
- Attempt probabilistic recovery or LLM-assisted correction.
- Produce human-readable postmortems.
- Stream-process live traces in the MVP.

Later versions may add streaming ingestion, partial recovery modes, richer schema negotiation, or adapter-specific prevalidation. None of that belongs here yet.

---

6. Responsibilities

What this module owns

- Input decoding from supported local sources.
- JSON parsing.
- Structural validation of the generic trace envelope.
- Field-level type validation.
- Semantic validation of trace invariants that are unambiguous and deterministic.
- Diagnostics generation with stable ordering and source references.
- Returning either a validated trace document or a failure result.

What it must never own

- Canonical model creation.
- Business logic about failures.
- Cross-event inference beyond explicit contract checks.
- Framework-specific adapter behavior.
- Output formatting for reports or UI.
- Error recovery through hidden heuristics.
- Any state that leaks between runs.

---

7. Inputs

Expected inputs

For the MVP, the module accepts one generic JSON trace format from local sources such as:

- a file path
- stdin
- a raw string already loaded in memory
- a byte buffer provided by the CLI or library caller

The input is expected to represent one execution run, not a batch of runs.

Validation rules

At minimum, the input must satisfy these structural rules:

- The top-level payload must be a JSON object.
- The payload must contain the required trace envelope fields defined by the project contract.
- The trace must contain a collection of events or steps that can be deterministically ordered.
- Each event must have the minimum identity and sequencing fields required by the contract.
- Any timestamps must be parseable and comparable.
- Any identifiers used for parent-child or causal references must be unique within the document.
- Unknown fields are allowed unless they conflict with reserved names or types.

Assumptions

- Input is UTF-8 encoded or convertible to UTF-8 without ambiguity.
- The document is self-contained for MVP purposes.
- The full input fits in memory on a developer laptop.

- The caller provides a single trace document at a time.
- The generic JSON schema is stable within a release line.

Malformed input handling

The module must fail fast on fatal issues such as:

- invalid JSON syntax
- unsupported encoding
- non-object top-level value
- missing required trace fields
- wrong value types for required fields
- duplicate identifiers
- unrecoverable timestamp parsing failures

For non-fatal issues, the module may emit warnings if the document can still be safely processed. Warnings must never silently downgrade a fatal structural error.

Malformed input must produce diagnostics with:

- severity
- machine-readable code
- JSON pointer or field path
- human-readable message
- stable ordering

---

8. Outputs

Exact outputs

The module returns one of two outcomes:

Success

A validated trace document containing:

- the parsed JSON structure
- source metadata
- normalized validation metadata
- an ordered diagnostics list that may include warnings
- any preserved unknown fields

Failure

A failure result containing:

- no validated trace document

- a complete ordered diagnostics list
- a clear fatal/non-fatal classification

Data artifacts

The output should expose the following internal artifacts to downstream stages:

- parsed JSON tree
- trace envelope metadata
- ordered event collection
- validation status
- diagnostic list
- source location map where available

Human-readable artifacts if relevant

This module does not own report generation, but it should provide diagnostics that the CLI can print in a concise human-readable form. That human-readable formatting belongs to the presentation layer, not here.

---

9. Internal Design

Main subcomponents

A minimal implementation should have five internal subcomponents:

1. **Input Reader**
   - Accepts file, stdin, bytes, or string.
   - Converts input into a UTF-8 text buffer with stable source metadata.
2. **JSON Decoder**
   - Parses the text buffer into a JSON object tree.
   - Captures line and column information when available.
3. **Structural Validator**
   - Verifies the top-level document shape.
   - Checks required fields and field types.
   - Confirms that collections exist where expected.
4. **Semantic Validator**
   - Validates cross-field invariants that are explicit and deterministic.
   - Checks uniqueness, ordering constraints, and legal references.
5. **Diagnostics Assembler**
   - Collects parse and validation issues.
   - Sorts diagnostics deterministically.
   - Produces the final success or failure result.

Workflow

```
Raw input
  |
  v
Input Reader
  |
  v
JSON Decoder
  |
  v
Structural Validator
  |
  v
Semantic Validator
  |
  v
Diagnostics Assembler
  |
  +--> Validated Trace Document
  |
  +--> Failure Result
```

Data flow

- The raw input is read once.
- The parser produces a JSON tree and source metadata.
- Structural validation runs before semantic validation.
- Semantic validation runs only if the structure is valid enough to support it.
- Diagnostics are accumulated, not thrown away.
- The final result is immutable.

Important internal decisions

- Validation should be explicit rule-by-rule, not hidden inside generic schema magic.
- Diagnostics should be structured objects, not plain strings.
- Ordering of diagnostics must be stable and testable.
- The module should preserve unknown fields rather than stripping them.
- The module should not attempt to "fix" the input.

---

10. Data Model

The module should work with a small set of internal entities.

Entity	Purpose	Notes
RawInput	Carries bytes or text plus source metadata	Used before decoding
ParsedJsonDocument	In-memory JSON tree plus source map	Produced by the decoder
TraceEnvelope	Root-level metadata for one execution run	Must be validated before downstream use
TraceEvent	One discrete event or step in the run	Stored in input order and reference order
ValidationIssue	Structured diagnostic entry	Used for errors and warnings
ValidationReport	Ordered collection of issues plus overall status	Returned for every parse attempt
ValidatedTraceDocument	Parsed and validated trace payload	The only output accepted by the normalizer

Key fields

The exact field names are defined by the project trace contract, but the implementation must treat the following concepts as first-class:

- document identity
- trace version
- run identity
- ordered event sequence
- event identity
- event timestamp or equivalent ordering marker
- parent or reference links where present
- raw metadata and unknown fields
- diagnostic provenance

Relationships

- One RawInput produces one ParsedJsonDocument or one failure.
- One ParsedJsonDocument produces one ValidationReport.
- A successful ValidationReport produces one ValidatedTraceDocument.
- One ValidatedTraceDocument maps to one input trace document.

Why this model exists

The model keeps the parser honest without dragging canonical-model complexity into the input stage. It separates "what the user gave us" from "what the system can safely trust."

---

11. Algorithms / Processing Logic

Step-by-step processing

1. Read the input from file, stdin, string, or bytes.
2. Convert to text using the agreed encoding rules.
3. Parse the text into a JSON tree.
4. Confirm the top-level value is an object.
5. Validate required root fields.
6. Validate the event collection.
7. Validate each event's required fields and types.
8. Validate deterministic invariants such as uniqueness and reference integrity.
9. Collect warnings and errors in a stable order.
10. Return a validated document only if no fatal issues exist.

Edge cases

- **Empty input**: fatal error.
- **Whitespace only**: fatal error.
- **Malformed JSON**: fatal error with line and column where possible.
- **Valid JSON, wrong root type**: fatal structural error.
- **Missing optional fields**: allowed unless required by the contract.
- **Duplicate event IDs**: fatal error.
- **Out-of-order events**: warning or fatal depending on whether the contract explicitly requires order; do not silently reorder.
- **Unknown fields**: preserved and ignored by validation unless they conflict with reserved names.
- **Mixed timestamp formats**: reject if they cannot be compared deterministically.
- **Partial files or truncated logs**: fatal error.

Decision rules

- If a rule is about syntax or contract shape, it is structural validation.
- If a rule depends on two or more fields agreeing, it is semantic validation.
- If a value cannot be interpreted deterministically, the module must fail rather than guess.

- If a warning would let bad data influence downstream analysis, promote it to an error.
- If an issue can be reported with a source location, report it.

Deterministic behavior requirements

- The same input must always yield the same diagnostics.
- Diagnostic ordering must not depend on hash map iteration or runtime nondeterminism.
- Reference resolution must use explicit rules only.
- Validation must not vary by machine locale or system timezone.
- Timestamps, if parsed, must be normalized into a single comparable representation before comparison.

---

12. Tradeoffs

Simplicity vs flexibility

The correct choice for the MVP is simplicity. A narrow, explicit input contract is easier to validate, easier to test, and easier to preserve across future changes. Supporting every weird trace shape now would just manufacture complexity and call it "extensibility."

Generality vs specificity

This module should be specific to the project's MVP trace contract, not generic JSON ingestion in the abstract. Over-generalization would force downstream modules to carry compatibility baggage they do not need.

Determinism vs heuristic inference

Determinism wins. If the input is ambiguous, this module should fail with a useful diagnostic rather than infer meaning from context. Hidden heuristics make failures harder to reproduce and undermine trust in the output.

Why the chosen design is preferred

This design creates a clean trust boundary. It gives the rest of the pipeline a stable, validated input while keeping the parser and validator small enough to reason about and test thoroughly.

---

13. Risks and Failure Modes

Hidden complexity

- Timestamps often look simple until they are not.
- Unknown fields can become accidental dependencies if they are not preserved thoughtfully.
- Event ordering can become ambiguous if producers do not guarantee stable sequence semantics.
- Location-aware diagnostics can become hard to maintain if the parser loses source offsets.

Ambiguous cases

- Multiple fields may seem to identify the same thing.
- An event may contain partial data that is structurally valid but semantically incomplete.
- A trace may be valid JSON but invalid as an execution record.
- The same input may have both errors and warnings; the error policy must remain unambiguous.

Unsupported inputs

- Multi-run bundles.
- Streaming live traces.
- Non-JSON trace formats.
- Framework-specific raw traces that have not been adapted into the generic JSON contract.
- Partially redacted documents that break field invariants.

Maintenance risks

- Turning validation rules into a config jungle.
- Letting adapter quirks leak into core input validation.
- Silently relaxing rules to make tests pass.
- Accumulating special cases for one-off traces instead of tightening the contract.

---

14. Testing Strategy

Unit tests

Unit tests should cover:

- JSON decoding success and failure

- encoding edge cases
- root object validation
- required field checks
- event field checks
- duplicate identifier detection
- timestamp parsing and comparison
- deterministic diagnostic ordering
- warning vs error classification

Integration tests

Integration tests should cover the full parser + validator flow using real fixture files.

Each integration case should verify:

- the final success or failure status
- the exact diagnostic codes
- the exact field paths or JSON pointers
- stable ordering of diagnostics
- preservation of unknown fields on success

Fixture-driven tests

Use a curated fixture set with named examples such as:

- valid minimal trace
- valid trace with unknown fields
- missing required root field
- duplicate event id
- malformed JSON
- non-object root
- invalid timestamp
- partially truncated document
- out-of-order events

Fixtures should be treated as contracts, not disposable test junk.

Regression protection

Any bug that affects parse validity, diagnostics, or source mapping must gain a fixture. The parser is a trust boundary; regressions here are not cosmetic.

---

15. Implementation Checklist

1. Define the exact MVP trace input contract.

2. Specify the structured diagnostic schema.
3. Implement deterministic input reading and encoding handling.
4. Implement JSON decoding with source metadata capture.
5. Add structural validation for root and event collection.
6. Add semantic validation for identifier uniqueness and reference integrity.
7. Preserve unknown fields and source metadata.
8. Normalize timestamp comparison into one internal representation.
9. Ensure validation result objects are immutable.
10. Add fixture-based tests for every error class.
11. Add integration tests for success and failure paths.
12. Verify diagnostic ordering is stable across repeated runs.
13. Document the contract for downstream modules and CLI users.
14. Refuse to add any recovery heuristic that is not explicitly justified.

---

16. Future Expansion

The following expansions naturally follow from this module, but they do **not** belong in the MVP:

- Schema version negotiation between trace producers and the parser.
- Streaming parsing for very large traces.
- Source span enrichment for richer CLI and UI highlighting.
- Adapter-specific prevalidation layers before generic validation.
- Optional lenient mode for exploratory debugging.
- Support for other trace transport formats once the generic JSON contract is stable.
- Incremental parsing for live capture workflows.

These are reasonable later-stage improvements only after the core path is stable.

---

17. Final Recommendation

This design is MVP-ready as long as the trace contract stays narrow and explicit. The module should remain strict, deterministic, and boring in the best possible way. Boring parsers are good parsers. They fail clearly, they test well, and they do not hallucinate meaning into broken input.

The only simplification still worth keeping on the table is this: keep the validation rule set small and explicit for v1, and do not introduce a configuration-driven rule engine unless the trace format actually starts changing frequently. Premature flexibility here would be a self-inflicted wound.